

SQLAlchemy

What is SQLAlchemy?

SQLAlchemy is a Python library that helps you work with databases using Python code instead of writing raw SQL everywhere. It provides both:

1. **Core** – SQL generation and database access.
2. **ORM (Object Relational Mapper)** – Maps Python classes to database tables.

Think of it like this:

Without SQLAlchemy

```
CREATETABLE users (  
    idINTEGERPRIMARYKEY,  
    name TEXT,  
    email TEXT  
);
```

Insert data:

```
INSERTINTO users(name,email)  
VALUES('John','john@gmail.com');
```

With SQLAlchemy ORM

```
classUser(Base):  
    __tablename__="users"  
  
    id=Column(Integer,primary_key=True)  
    name=Column(String)  
    email=Column(String)
```

Now Python objects become database rows.

```
user=User(  
    name="John",
```

```
email="john@gmail.com"  
)
```

SQLAlchemy converts that into SQL behind the scenes.

Why Do We Use SQLAlchemy?

Imagine you are building:

- Employee Management System
- Student Portal
- E-commerce Website
- RAG Application User Management

Without SQLAlchemy:

```
cursor.execute(  
    "INSERT INTO users(name,email) VALUES (?,?)",  
    ("John","john@gmail.com")  
)
```

With SQLAlchemy:

```
db.add(user)  
db.commit()
```

Cleaner and easier to maintain.

SQLAlchemy Architecture

```
Python Class  
  ↓  
SQLAlchemy ORM  
  ↓  
SQL Query  
  ↓  
Database
```

Example:

```
class User(Base):  
    __tablename__ = "users"  
  
    id = Column(Integer, primary_key=True)  
    name = Column(String)
```

Table created:

```
users  
-----  
id  
name
```

Step 1: Install

```
pip install sqlalchemy
```

For FastAPI:

```
pip install sqlalchemy fastapi uvicorn
```

Step 2: Create Database Connection

```
from sqlalchemy import create_engine  
  
DATABASE_URL = "sqlite:///users.db"  
  
engine = create_engine(DATABASE_URL)
```

This connects Python to SQLite.

Step 3: Create Base

```
from sqlalchemy.orm import declarative_base
```

```
Base=declarative_base()
```

Every table will inherit from `Base`.

Step 4: Create Model

```
from sqlalchemy import Column,Integer,String

class User(Base):
    __tablename__="users"

    id=Column(Integer,primary_key=True)
    name=Column(String)
    email=Column(String)
```

This Python class becomes a database table.

Step 5: Create Table

```
Base.metadata.create_all(bind=engine)
```

Database:

```
users
-----
id
name
email
```

Step 6: Create Session

A Session is like a database cursor.

```
from sqlalchemy.orm import sessionmaker

SessionLocal=sessionmaker(bind=engine)

db=SessionLocal()
```

Step 7: Insert Data

```
new_user=User(
    name="Sayyami",
    email="sayyami@gmail.com"
)

db.add(new_user)
db.commit()
```

Generated SQL:

```
INSERT INTO users
(name,email)
VALUES
('Sayyami','sayyami@gmail.com');
```

Step 8: Read Data

```
users=db.query(User).all()

for user in users:
    print(user.name)
```

Generated SQL:

```
SELECT*FROM users;
```

Step 9: Filter Data

```
user=db.query(User).filter(
    User.name=="Sayyami"
).first()

print(user.email)
```

Generated SQL:

```
SELECT*  
FROM users  
WHERE name='Sayyami';
```

Q: Why do we use `connect_args={"check_same_thread": False}` in FastAPI with SQLite?

A: SQLite restricts a database connection to the thread that created it. FastAPI can handle requests across multiple threads, so setting `check_same_thread=False` allows the SQLite connection to be accessed from different threads and prevents thread-related database errors.